Lecture 5 · Mon 10/12/2026

CS160 Compilers — Fall 2026

Lexing.

Yu Feng

PA2 is out — this lecture and the next two *are* PA2.

Where we are.

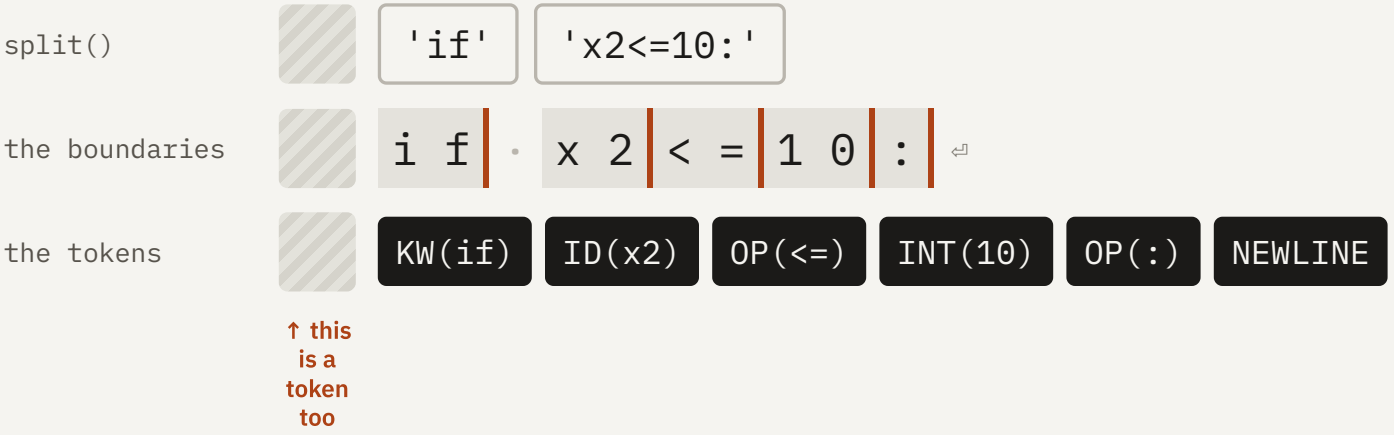


So far `ast.parse` did the first two arrows for free.

PA2: you write them.

Today the first arrow — the easiest part of a compiler, with the most theory.

It looked like `split()`. It isn't.



x2 is one word, not two. <= is one token, not two — and no space tells you where either one ends.

`if` looks exactly like a variable name; nothing in the characters says otherwise.

And in Python the *indentation* carries meaning that isn't any character you can see.

The real job: an infinite set of strings has to become a short, finite list of *kinds* — and everything downstream works only because that list is finite.

How do you turn a stream of characters into the words of a program?

By the end you know what a regular expression, an NFA and a DFA each *are*, and can read any of the three.

You can turn a regular expression into an NFA and that NFA into a DFA, on paper, and say how many states you get.

You can run a DFA across an input string and mark where each token ends.

All pencil-and-paper — they are Midterm 1, a week from today. PA2 is the same thing in code, and it's out today.

A token is a *kind* plus a *lexeme* plus a *position*.

```
source:   while i < n:

tokens:   KW(while)  ID(i)  OP(<)  ID(n)  OP(:)  NEWLINE
          3:1        3:7    3:9    3:11   3:12   3:13
```

kind

which of the eight classes this is — `KW`, `ID`, `OP`, …

lexeme

the exact characters the token was made of: `while`, `i`, `<`

position

line and column of its first character.

The lexer drops spaces and comments and keeps what the parser will ask about: what kind, which characters (the *lexeme*), where.

Eight kinds. The whole lexical language of ChocoPy through PA3.

```
# ChocoPy v2.2 §3.3 — the keyword list is exactly Python 3's
KEYWORDS  = {"False", "None", "True", "and", "as", "assert", "async", "await",
             "break", "class", "continue", "def", "del", "elif", "else",
             "except", "finally", "for", "from", "global", "if", "import",
             "in", "is", "lambda", "nonlocal", "not", "or", "pass", "raise",
             "return", "try", "while", "with", "yield"}
# ChocoPy v2.2 §3.5 — operators and delimiters; multi-character ones first
OPERATORS = ["==", "!=", "<=", ">=", "//", "->",
             "+", "-", "*", "%", "<", ">", "=",
             "(", ")", "[", "]", ",", ":", "."]
# plus: ID  [A-Za-z_][A-Za-z0-9_]*  §3.2      INT  0|[1-9][0-9]*  §3.4.2
# plus: NEWLINE  INDENT  DEDENT  EOF     <- the ones Python needs and C doesn't
```

The keywords we don't use yet are **reserved**: the lexer still emits `KW` for them, so they can never be identifiers. And `print` is an ordinary `ID`, exactly as in Python 3 — §3.3 has no `print` in it.

Every token kind is a regular expression.

regular expression a pattern that names a *set* of strings. `[0-9]+` is not one number; it is every number.

```
ID   [A-Za-z_][A-Za-z0-9_]*                §3.2
INT  0 | [1-9][0-9]*                       §3.4.2   max 2**31 - 1
OP   an alternation of 20 literals         §3.5
```

An identifier is a letter or underscore, then any number of letters, digits and underscores.

The lexer's job: at the cursor, which pattern matches, and how far?

A pattern is a description. To *run* it we need a machine — and the next ten slides build that machine from the pattern.

Three operators. Everything else is sugar.

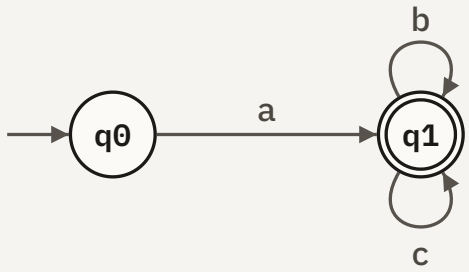
```
a          the character a
r1 r2      r1 then r2                  (concatenation)
r1 | r2    r1 or r2                    (alternation)
r*         zero or more r              (Kleene star)
sugar:  r+ = r r*      r? = (r | ε)      [a-c] = (a|b|c)
```

ε the empty string — the pattern that matches *no characters at all*. So `r?` reads “r, or nothing”.

Today's example uses all three: `a(b|c)*` — an a, then any number of bs and cs. It matches a, ab, acb, abbc; it does not match b, ba or the empty string.

A DFA reads one character at a time and is always in exactly one state.

DFA *deterministic finite automaton*: finitely many states, a start state, some accepting states, and from each state at most one arrow per character.



This machine accepts exactly the strings of `a(b|c)*`. One step per character, no choices, no going back — which is why a lexer wants a DFA.

We drew it by hand. The rest of this segment gets it *mechanically* from the pattern, in two steps.

```
input  a b c b      q0 -a-> q1 -b-> q1 -c-> q1 -b-> q1     ends in q1, accepting: ACCEPT
input  b a         q0 -b-> no arrow                        the machine dies:      REJECT
```

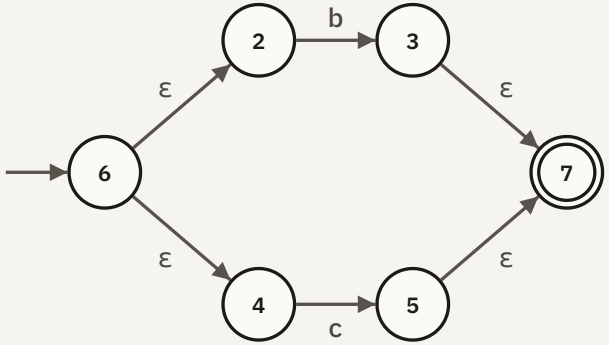
An NFA may be in several states at once.

NFA

nondeterministic finite automaton: like a DFA, but a state may have several arrows for the same character, or none.

ε-transition

an arrow the machine may follow **without reading a character**. On slide 8 ε was the empty string; an arrow labelled ε consumes nothing, so it is a free move.



Don't guess which arrow to take: follow *all* of them and keep the set of states the machine could be in. It accepts if that set contains an accepting state at the end.

NFAs are easy to build from a pattern and awkward to run. DFAs are easy to run and awkward to build. So: **pattern → NFA → DFA**.

```
before reading anything     {6}  free moves  {6, 2, 4}
read b                      only state 2 has a b arrow: {3}  free moves  {3, 7}
end of input                7 is accepting: ACCEPT
```

Both are five things. They differ in one.

	NFA = (Q, Σ, δ, q0, F)	DFA = (Q, Σ, δ, q0, F)
Q	a finite set of states	(same)
Σ	the alphabet: the input's characters	(same)
q0	the start state, q0 in Q	(same)
F	the accepting states, F subset of Q	(same)
δ	δ : Q × (Σ ∪ {ε}) → P(Q) a SET of next states: maybe none, maybe several, maybe on ε (read nothing)	δ : Q × Σ → Q exactly ONE next state, never on ε
	accepts a string if SOME path from q0 reading it ends in F	accepts if THE path from q0 reading it ends in F

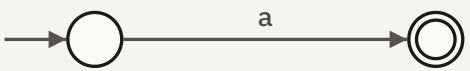
Four rows identical; the whole difference is δ. In the DFA it is a *function* — one (state, character), one answer — and that is what *deterministic* means.

Slide 9 was the right-hand column as a picture; slide 10 was the left-hand column.

Step 1 — Thompson's construction

Regex → NFA, one rule per operator.

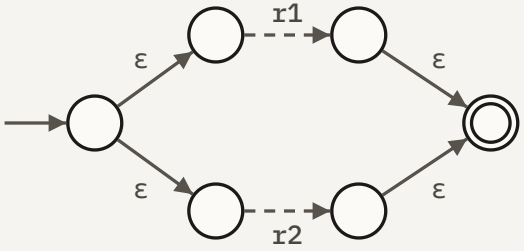
a



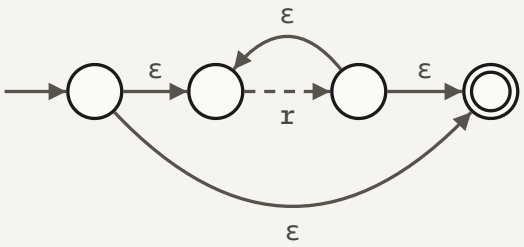
r1 r2



r1 | r2



r*



Thompson's construction

the recipe for step 1: build a small NFA for each character, then glue the small ones together following the pattern's operators.

------- a whole piece, start to accept

Every piece has one start and one accept, and pieces connect only by ε edges. That is why the rules compose: any pattern, no thinking.

Step 1, worked: a(b|c)*

Build it inside-out, one operator at a time.






```
1.  b  and  c          2 --b--> 3           4 --c--> 5
```

10 states and 9 ε edges for a 7-character pattern. Thompson is wasteful on purpose: simple rules, no cleverness. Step 2 cleans up.

Step 2 — subset construction

NFA → DFA: a DFA state is a *set* of NFA states.

subset construction

the recipe for step 2, and its whole idea: one DFA state *is* the set of states the NFA could be in right now — exactly the sets we wrote down on slide 10.

```
D0 = ε-closure({start})
for each DFA state S and each character c:
    T = ε-closure( move(S, c) )
    if T is empty: no arrow.   otherwise: add T if it is new, and add the arrow S --c--> T
accepting = every S that contains the NFA's accept state
```

move(S, c)

the states you reach from any state in S by one arrow labelled c.

ε-closure(S)

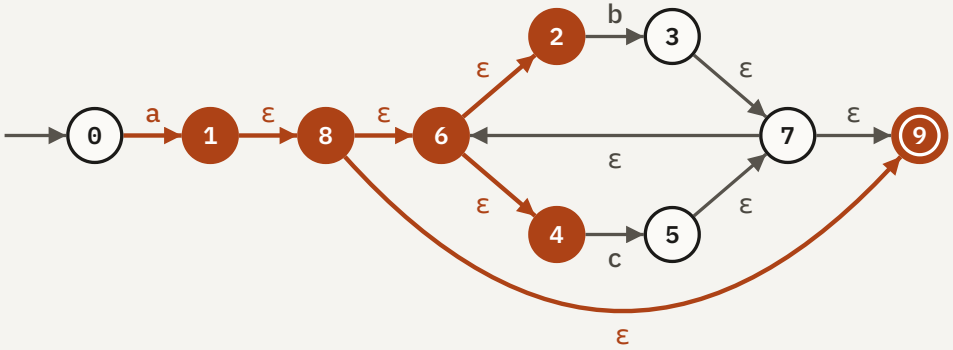
S plus everything reachable from it by ε arrows alone — the free moves of slide 10.

Repeat until no new sets appear. The NFA has finitely many states, so there are finitely many sets, so it stops.

Step 2, worked: the first two states

Start where the NFA starts. Ask one question per character.

the slide-13 NFA · ε-closure({1}) lit



the DFA so far




```
D0 = ε-closure({0})              = {0}                    no ε arrows leave 0

from D0 on a:   move = {1}       ε-closure = {1, 8, 6, 2, 4, 9}     new state  D1
from D0 on b:   move = {}        no arrow
from D0 on c:   move = {}        no arrow

D1 = {1, 2, 4, 6, 8, 9}          contains 9, the NFA's accept state:  D1 is ACCEPTING
```

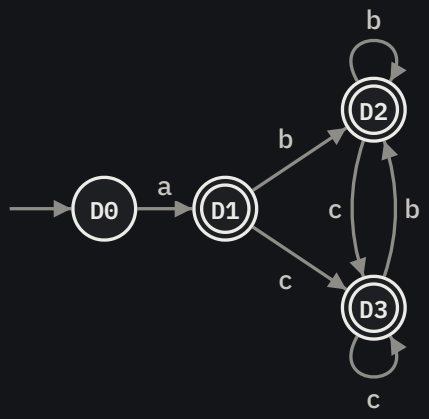
Follow the ε-closure of {1} with a finger: 1 → 8, then 8 → 6 and 9, then 6 → 2 and 4. Six NFA states, one DFA state.

Step 2, worked: finish the table

Keep going until nothing new appears.

```
from D1 on b:   move = {3}       ε-closure = {3, 7, 6, 9, 2, 4}     new state  D2
from D1 on c:   move = {5}       ε-closure = {5, 7, 6, 9, 2, 4}     new state  D3
from D2 on b:   move = {3}       that is D2 again
from D2 on c:   move = {5}       that is D3 again
from D3 on b:   D2               from D3 on c:   D3                  nothing new: stop

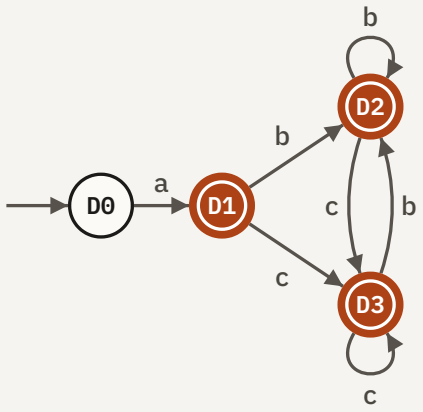
D0 = {0}                                 D0 --a--> D1
D1 = {1, 2, 4, 6, 8, 9}  ACCEPT          D1 --b--> D2   D1 --c--> D3
D2 = {2, 3, 4, 6, 7, 9}  ACCEPT          D2 --b--> D2   D2 --c--> D3
D3 = {2, 4, 5, 6, 7, 9}  ACCEPT          D3 --b--> D2   D3 --c--> D3
```



10 NFA states became 4 DFA states, and no ε anywhere. Read them: D1 = “just read the a”, D2 = “last character was b”, D3 = “last character was c”.

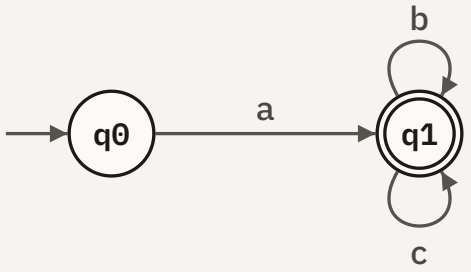
D1, D2 and D3 are triplets.

slide 16 · 4 states



merge →

2 states · slide 9's machine



Two states are *equivalent* if no input can tell them apart from here on.

D1, D2, D3: all accepting; on b all three go to D2; on c all three go to D3; on a all three die. Nothing distinguishes them, so merge them.

Result: `q0 --a--> q1`, with q1 looping on b and c. Two states — the machine we drew by hand on slide 9.

Pattern → NFA → DFA → smallest DFA, and not one step needed an idea. Now read a as “letter” and b | c as “letter or digit”: this is the machine for `ID`.

Your turn: ab|a

Regex → NFA → DFA, on paper.

Build the NFA for `ab|a` with Thompson's rules.

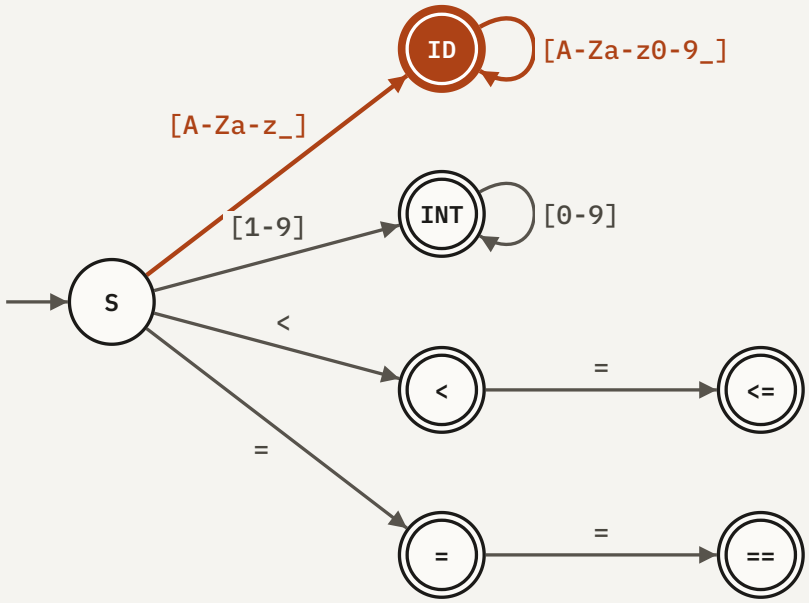
Run the subset construction on it.

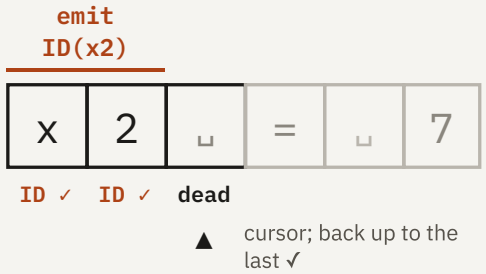
How many DFA states, and which are accepting?

Longest match wins.

maximal munch the rule's name: at each position take the *longest* string any pattern matches — never the first pattern that happens to match.

`ID, INT, <, <=, =, == merged into one DFA (a subset)`





Run the DFA as far as it goes; remember the last accepting position; emit that token, reset, continue.

x2 is one `ID` and <= is one `OP` *because* the machine kept going.

Keywords need no machine of their own: `if` matches the `ID` pattern, so lex it as an identifier, then look it up in the keyword table.

C has braces. Python has whitespace.

indent stack a stack holding the indentation width of every block currently open. It starts as `[0]`.

line	spaces	stack	emitted first
A	0	[0]	
B	4	[0, 4]	**INDENT**
C	8	[0, 4, 8]	**INDENT**
D	4	[0, 4]	**DEDENT**
E	0	[0]	**DEDENT**

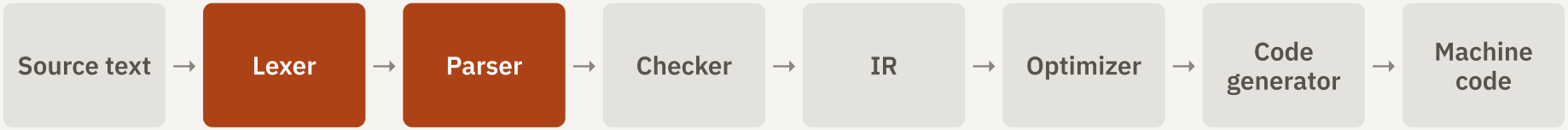
Width **equal** to the top: nothing.

Width **larger**: push it, emit one INDENT.

Width **smaller**: pop every larger number, emit one DEDENT per pop.

The lexer manufactures the braces the parser wants: INDENT = {, DEDENT = }, NEWLINE = ;. A DFA cannot count how many blocks are open, so the lexer keeps this one small stack beside it.

PA2



Out today, due Wed 10/21. Lexer (today) + parser (Wed, and the Wed after Midterm 1) for PA1's subset of ChocoPy, producing Python `ast` nodes.

Oracle: `ast.dump(yours) == ast.dump(ast.parse(src))`. Your lecture-3 compiler runs unchanged on your tree.

Section Thursday: PA2 lab, bring your lexer.

Wednesday: tokens → tree.

Grammars, ambiguity, why `1 - 2 - 3` is two trees and only one is right.

Recursive descent: one function per rule — a working expression parser by the end of class.

